EXP.NO: 5	Write a Servlet to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.
DATE:20/02/2026	

AIM:

To demonstrate the difference between the HTTP GET and POST methods by creating a form in an HTML page and handling the form submissions using two different HTTP methods in a Java Servlet.

ALGORITHM:

1. Input:

- The user fills out a web form with their **name** and **email**.
- The form will use both **GET** and **POST** methods for data submission.

2. Steps for Creating and Handling the HTTP GET and POST Requests:

1. **Create an HTML Form:**
    - Design an HTML form with two input fields: one for the **name** and one for the **email**.
    - Create two buttons to allow the user to submit the form using **GET** or **POST** methods.
    - Use the action attribute to define the **Servlet URL** and specify the form method (either GET or POST).
2. **Create the Servlet to Handle GET and POST Requests:**
    - Write a Java Servlet class that will handle **both GET and POST requests**.
    - The servlet will have two methods:
        - doGet() to handle the GET requests.
        - doPost() to handle the POST requests.
    - In both methods, retrieve the form data using request.getParameter().
    - In the doGet() method, process and display the data in the URL.

- In the doPost() method, process and display the data in the HTTP request body.

3. **Handle GET Requests in the Servlet:**
   - In the doGet() method, use request.getParameter() to retrieve the name and email submitted via GET.
   - Output the data as a response, including the **name** and **email** in the URL (visible to the user).
4. **Handle POST Requests in the Servlet:**
   - In the doPost() method, use request.getParameter() to retrieve the name and email submitted via POST.
   - Output the data as a response, displaying the **name** and **email** without exposing them in the URL (more secure).
5. **Return the Output:**
   - The Servlet should return an HTML page displaying the **name** and **email** entered by the user.
   - For the **GET method**, the output should be visible in the URL.
   - For the **POST method**, the output should be visible in the response body, not the URL.
6. **Deploy and Test the Servlet:**
   - Compile the Servlet class and deploy it to a **Servlet container** (like **Apache Tomcat**).
   - Test the form using **GET** and **POST** methods to see the difference in data submission.
7. **Observe and Compare GET and POST Results:**
   - Test the **GET method**: The form data is sent in the URL.
   - Test the **POST method**: The form data is sent in the HTTP request body, not visible in the URL.

CODE:

```
<!DOCTYPE html>
<html>
<body>

<form action="MethodServlet" method="get">
    Name:
    <input type="text" name="name">
    <input type="submit" value="GET">
</form>

<br>

<form action="MethodServlet" method="post">
    Name:
    <input type="text" name="name">
    <input type="submit" value="POST">
</form>

</body>
</html>
```

MethodServlet.java:

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
```

```
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/MethodServlet")
public class MethodServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
            throws IOException {

        PrintWriter out=response.getWriter();

        String name=request.getParameter("name");

        out.println("<h2>GET Method Called</h2>");
        out.println("Name: "+name);
    }

    protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
            throws IOException {

        PrintWriter out=response.getWriter();

        String name=request.getParameter("name");
```

```
        out.println("<h2>POST Method Called</h2>");

        out.println("Name: "+name);

    }
```

OUTPUT:

RESULT:

A Servlet is created to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.